

MetaGPT-Mini

An LLM-agnostic reimplementation of
MetaGPT (ICLR 2024 Oral, #1 LLM-Agent)

A learning guide for the M.Tech class

Jai Kumar Meena - M.Tech by Research, CSE
Delhi Technological University

<https://github.com/mannuking/metagpt-mini>

Five papers. One implemented. Zero closed-source models.

1. Why I Picked MetaGPT Out of the Five

I had to choose **one paper** to teach in class tomorrow out of five famous agentic-engineering papers, all from top-tier venues (NeurIPS, ICLR, COLM). Here is the comparison and the reasoning.

The five papers at a glance

MetaGPT - Multi-agent orchestration with SOPs. PM, Architect, Engineer, QA collaborating via a shared message pool. (*ICLR 2024 Oral - top 1.2% of all submissions, ranked #1 in the LLM-Agent category.*)

SWE-agent - Agent-Computer Interfaces for autonomous software engineering. State-of-the-art on SWE-bench at the time of publication. (*NeurIPS 2024.*)

OpenHands (OpenDevin) - Production-grade agent runtime. Terminals, browsers, editors, code interpreters in one harness. (*COLM 2024 Oral.*)

Reflexion - Self-improving agent memory via verbal reinforcement. (*NeurIPS 2023.*)

Voyager - Lifelong skill acquisition: an agent that grows its own tool library. (*NeurIPS 2023.*)

Why MetaGPT won

Pedagogical completeness. MetaGPT is the only paper that models an entire organization (PM + Architect + Engineer + QA). Five other layers (orchestration, message passing, role specialization, SOP encoding, output verification) all appear in one paper. **Reproducibility in 1 hour.** The data flow is message-passing, not heavy machinery like Docker (OpenHands), reinforcement-learning training loops (Reflexion), or skill-library code-execution sandboxes (Voyager). A student can read and implement MetaGPT in 1 hour; the others take 1 day. **Real citations.** 50k+ GitHub stars on the original, hundreds of follow-up papers, ICLR Oral - no marketing fluff needed. **Maps to the 2026 frontier.** The harness-engineering papers I researched (arxiv 2603-2609) all explicitly cite MetaGPT as the multi-agent baseline. Teaching MetaGPT is teaching the foundation the new 2026 work builds on.

What MetaGPT does NOT do (where the others win)

- SWE-agent beats it on raw SWE-bench scores (better agent-computer interface).
- OpenHands is the production-grade runtime that would actually deploy MetaGPT.
- Reflexion adds self-reflection - MetaGPT does not learn from past runs.
- Voyager grows skills continuously - MetaGPT is stateless across runs.

All of those are *additive*: you can bolt Reflexion self-reflection or Voyager skill library onto MetaGPT. MetaGPT is the substrate.

2. The Four Other Papers - Context

Even though MetaGPT is what we implement, you should be able to place it next to its peers on a slide and answer: why not the other? Here is the at-a-glance summary of each.

Paper	Venue	Year	Core idea	Best demo time
MetaGPT	ICLR 2024 Oral	2024	Multi-agent SOP orchestration (PM/Arch/Eng/QA)	1 minute
SWE-agent	NeurIPS 2024	2024	Agent-Computer Interface design pattern	5 minutes (needs Docker)
OpenHands	COLM 2024 Oral	2024	Production-grade agent harness runtime	10+ min (full Docker stack)
Reflexion	NeurIPS 2023	2023	Verbal-RL: agents self-reflect from failure	1 minute
Voyager	NeurIPS 2023	2023	Lifelong skill acquisition + curriculum	5+ min (needs Minecraft env)

Each paper in 60 seconds

SWE-agent (NeurIPS 2024)

Yang et al. noticed that the LLM is fine, but the *interface* matters more. A custom Agent-Computer Interface (ACI), with short commands and scoped outputs, lets GPT-4 solve 12.29% of SWE-bench issues end-to-end. The key insight: do not make the LLM type shell commands; give it purpose-built verbs.

OpenHands (COLM 2024 Oral)

Formerly OpenDevin. A platform that runs an agent with: a bash sandbox, a browser, an editor, an IPython kernel, all under one runtime. This is what you would actually *deploy*; MetaGPT is what you would actually *reason about*. OpenHands builds a MetaGPT-like team but inside a real runtime.

Reflexion (NeurIPS 2023)

Shinn et al. let an agent keep a verbal *self-reflection* after every failed trial. Next time, it reads its own past reflections and decides differently. No weight updates - just text in a memory buffer. Drops onto MetaGPT like a memory layer.

Voyager (NeurIPS 2023)

Wang et al. dropped GPT-4 into Minecraft with three modules: a curriculum generator that proposes the next hardest achievable task, a skill library that stores reusable code, and a reflection module. The agent grows its own toolkit forever. This is the skills-as-a-knowledge-graph idea, conceptually close to the Hermes skill system.

The meta-takeaway

Every one of these papers can be summarized as: *give the LLM a better loop*. MetaGPT gives it a team. SWE-agent gives it better tools. OpenHands gives it a runtime. Reflexion gives it memory. Voyager gives it lifelong growth. The frontier of 2026 harness-engineering is putting all of these together.

3. The MetaGPT Paper - Read-Along Guide

If you are going to teach this paper to others, here is how to read it in one sitting. The PDF is in the project folder: *02_metagpt.pdf*.

Section 1 Introduction - what problem they solve

Existing single-agent LLM systems fail at complex software engineering because they have no division of labour and no standard procedures. Humans solve this with SOPs (PM writes specs, Architect designs, Engineer codes, QA tests). MetaGPT claim: encoding SOPs as prompt sequences and running them across role-specialised agents gets you 3.6x better and 10x cheaper than GPT-4 single-agent on HumanEval pass@1.

Section 2 Related Work - where MetaGPT sits

- Single-agent (AutoGPT, BabyAGI): no structure, explodes in cost
- Multi-agent role-play (CAMEL): two agents, no SOP
- Society of Mind (Minsky, 1986): theoretical, no LLM implementation
- MetaGPT is the first to put SOPs into a multi-agent LLM system

Section 3 Framework - the architecture we re-implement

The MetaGPT architecture has three layers:

- **Environment** - a shared space where messages are published and subscribed. (Our MessagePool in `schema.py`.)
- **Agents (Roles and Profiles)** - a Role has a name, a profile (Senior PM), and a set of Actions it can execute. (Our Role class in `roles.py`.)
- **Actions** - the verbs. `WritePRD`, `WriteDesign`, `WriteCode`, `WriteTest`. Each takes context, emits a Message.

The *message* object is the key abstraction. It carries `role`, `content`, `cause_by`, `msg_id`, `created_at`. Every agent sees every message but only acts on the ones its SOP needs.

Section 3.4 Standard Operating Procedures - the meta-pattern

An SOP in MetaGPT is the directed-acyclic-graph of actions. The canonical SOP is: *WritePRD then WriteDesign then WriteCode then WriteTest*. Our `DEFAULT_SOP` list and `make_canonical_team` in `roles.py` implement exactly this.

Section 4 Experiments - what MetaGPT was measured on

- **HumanEval pass@1**: 85.9% with GPT-3.5 vs 67.8% single-agent (cost reduced 10x)
- **HumanEval-ET**: higher-quality pass when full SOP team runs
- **MBPP**: similar pattern; SOP team consistently wins
- **Software development benchmark**: full project (3.6x quality of single-agent)

Section 5 Discussion - limitations and future

MetaGPT does not learn across runs (that is where Reflexion and Voyager extend it). It assumes humans provide the SOP; it does not invent one. These limits define exactly the open problems the 2026 harness-engineering papers are trying to solve.

4. Our Reimplementation - File-by-File Walkthrough

Every line in MetaGPT-Mini maps to a concept in the paper. Here is what each file does, why it exists, and how to read it. The codebase is intentionally small - 2,408 LOC of Python total - so a student can read the whole thing in one sitting.

metagpt_mini/llm.py - Anthropic Messages client (221 LOC)

The only file that knows about an LLM provider. Wraps the Anthropic /v1/messages Messages API with three call modes. Anything that exposes an Anthropic-compatible endpoint (**MiniMax-M3**, **Anthropic Claude**, local **Ollama**) works without code changes.

```
class LLM:
    def __init__(self, model=None, base_url=None, api_key=None,
                 temperature=None, max_tokens=None):
        self.model = model or os.getenv('LLM_MODEL', 'MiniMax-M3')
        self.base_url = base_url or os.getenv('LLM_BASE_URL', 'https://api.minimax.io/anthropic')
        self.api_key = api_key or os.getenv('LLM_API_KEY', '')
        self.context_window = int(os.getenv('LLM_CONTEXT_WINDOW', '1048576'))
        self.usage = LLMUsage() # running token + cost counter
        self._client = Anthropic(api_key=self.api_key, base_url=self.base_url)

    def chat(self, messages, *, role_tag='') -> str:
        # blocking single-turn call (text out)

    def stream(self, messages, *, role_tag='') -> Iterator[str]:
        # yields text chunks as the LLM emits them - drives the live TUI

    def structured(self, messages, *, tool_name, tool_description,
                  input_schema, role_tag='') -> dict:
        # Anthropic tool-use: forces the LLM to return JSON matching
        # input_schema. Used by WriteCode for multi-file manifests.

    def system_user(self, system, user, *, role_tag='') -> str:
        return self.chat([ChatMessage('system', system),
                          ChatMessage('user', user)],
                          role_tag=role_tag)
```

Key idea: LLMUsage is a running counter of input tokens, output tokens, cache hits, and estimated USD cost (using configurable `COST_INPUT_PER_1K` / `COST_OUTPUT_PER_1K`). Every chat/stream/structured call appends a per-role entry to `usage.log`. `Team.run` checks `usage_cost()` against `MAX_BUDGET_USD` between every role and stops if the budget is exceeded. The Mission Control TUI panel reads this log to show live per-role token + cost breakdowns.

metagpt_mini/schema.py - Message, MessagePool, Manifest (82 LOC)

The substrate. Every piece of state in MetaGPT is a Message in a MessagePool. Two small classes, but the entire system runs on them.

```
@dataclass
class Message:
    role: str # ProductManager, Engineer, ...
    content: str # free-form text
    cause_by: str = '' # WritePRD, WriteDesign, WriteCode, WriteTest
    msg_id: str = uuid.uuid4()[ :8] # short
    created_at: str = isoformat UTC
    round_num: int = 0 # which Engineer->QA iteration
    extra: dict = {} # file paths, manifest refs, ...
```

```

class MessagePool:
    def publish(self, msg): ...
    def history(self): ...
    def of_role(self, name): ... # messages from a role
    def by_action(self, cause_by): ... # messages from an action
    def latest_of(self, cause_by): ... # the most recent of one action

@dataclass
class Manifest:
    '''Engineer's structured output: a set of files to write to disk.'''
    project_name: str
    summary: str
    files: List[FileEntry] # path + content + rationale per file
    dependencies: List[str] # pip packages (stdlib if empty)
    run_instructions: str # how to run the generated project

```

Key idea: `cause_by` is the traceable SOP link. If you ever ask *why did the Engineer write this code?*, read the chain: `UserInput`, `WritePRD`, `WriteDesign`, `WriteCode`. The new `Manifest` type is the structured output of `WriteCode` - it carries the full multi-file project so it can be saved to disk after the run.

metagpt_mini/actions.py - The four verbs (443 LOC)

Each Action is a dataclass with one method: `run(pool, *, on_token=None, round_num=0)`. Read from the pool, call the LLM, write the result back.

```

@dataclass
class WritePRD:
    llm: LLM
    role_name = 'ProductManager'
    cause_by = 'WritePRD'

    def run(self, pool, *, on_token=None, round_num=0):
        out = ''
        for chunk in self.llm.stream([sys, user], role_tag='ProductManager'):
            out += chunk
            if on_token: on_token(chunk) # drives the live TUI
        msg = Message(role='ProductManager', content=out, cause_by='WritePRD')
        pool.publish(msg)
        return msg

# WriteDesign - same shape, reads WritePRD from pool.
# WriteCode - uses llm.structured() with tool_name='emit_manifest'
#             to GUARANTEE a JSON dict of files (no parsing roulette).
#             Returns (Message, Manifest). Has retry-if-truncated logic.
# WriteTest - uses llm.structured() with tool_name='qa_report' to
#             emit {passed, issues[], summary} - drives the QA loop.

```

Four actions in canonical SOP order: `WritePRD`, `WriteDesign`, `WriteCode`, `WriteTest`. **The big shift from the original MetaGPT:** `WriteCode` uses Anthropic tool-use (via `llm.structured()`) instead of JSON-parsing streamed text. This gives us a guaranteed JSON manifest for the multi-file project - no more regex-extracting `{...}` blocks from chat output.

metagpt_mini/roles.py - The four agents (55 LOC)

```

@dataclass
class Role:
    name: str # 'ProductManager'
    profile: str # 'Senior PM'
    color: str # for the UI - cyan / magenta / green / yellow
    actions: List[Callable] # factories: [lambda: WritePRD(llm)]
    action_names: List[str] # ['WritePRD'], etc.

    def run(self, pool, *, on_token=None, round_num=0, qa_feedback=None):
        for action_factory, name in zip(self.actions, self.action_names):

```

```

        action = action_factory()
        if name == 'WriteCode':
            msg, _ = action.run(pool, on_token=on_token,
                               round_num=round_num, qa_feedback=qa_feedback)
        else:
            msg = action.run(pool, on_token=on_token, round_num=round_num)
        # publish handled inside each action

def make_canonical_team(llm) -> List[Role]:
    '''PM -> Architect -> Engineer -> QA. The MetaGPT canonical SOP.'''
    return [
        Role('ProductManager', 'Senior PM', color='cyan',
             actions=[lambda: WritePRD(llm)], action_names=['WritePRD']),
        Role('Architect', 'Senior Architect', color='magenta',
             actions=[lambda: WriteDesign(llm)], action_names=['WriteDesign']),
        Role('Engineer', 'Senior Backend Engineer', color='green',
             actions=[lambda: WriteCode(llm)], action_names=['WriteCode']),
        Role('QA', 'Senior QA Engineer', color='yellow',
             actions=[lambda: WriteTest(llm)], action_names=['WriteTest']),
    ]

```

Key idea: The team is data, not code. To reconfigure the SOP you change `make_canonical_team` - reorder, add, remove roles - no other file needs to change. The `color` attribute is the role's identity in the TUI.

metagpt_mini/team.py - Orchestrator + live-mode runner (511 LOC)

Owns the MessagePool, holds the Roles, runs them in SOP sequence, manages the budget, and (in live mode) drives the full-screen TUI via a worker-thread animation loop.

```

@dataclass
class Team:
    llm: LLM
    pool: MessagePool = field(default_factory=MessagePool)
    roles: List[Role] = None
    live: bool = True

    def run(self, requirement, *, budget_usd=None, max_revisions=2):
        if self.live: return self._run_with_live(requirement, budget, max_revisions)
        return self._run_blocking(requirement, budget, max_revisions)

    def _run_with_live(self, requirement, budget, max_revisions):
        state = LiveRunState(...)
        with Live(render(state), refresh_per_second=8, screen=True) as live:
            state.live = live
            # PM runs once
            self._run_role_live(self.roles[0], round_num=1)
            # Architect runs once
            self._run_role_live(self.roles[1], round_num=1)
            # Engineer <-> QA loop (up to max_revisions rounds)
            for round_num in range(1, max_revisions + 2):
                self._run_engineer_live(self.roles[2], round_num, qa_feedback, live=live)
                qa_msg = self._run_qa_live(self.roles[3], round_num, live=live)
                if qa_msg.extra.get('passed'): break

```

Key idea: Each `_run_*_live` method runs the role in a **worker thread** so the main thread can keep the TUI spinner and progress bars ticking at 8 Hz while the LLM call blocks. When the structured call lands, `cancel_fake_progress()` snaps all bars to 100% and the real manifest view takes over. `_run_blocking` is the non-UI fallback used by tests and CI.

metagpt_mini/ui.py - Full-screen live TUI (623 LOC)

The new piece. Builds the 6-region rich Live layout you see in the demo: pyfiglet banner, role streaming panel, Mission Control stats, progress table, event log, and real-world apps footer. Also drives the

per-phase fake-progress animation when Engineer is in its tool-use call (no real tokens stream there).

```
@dataclass
class LiveRunState:
    role, action, color, phase, content
    manifest, messages, events (deque maxlen=50)
    tokens_in, tokens_out, elapsed_s, cost_usd
    # Per-role cost maps populated from llm.usage.log
    role_tokens_in, role_tokens_out, role_cost_usd
    # Fake-progress state (see below)
    _fake_file_plan, _fake_progress, _fake_tick

    def layout(self) -> Layout:
        # 6 regions: banner / main (role_panel + stats) /
        #                progress / log / apps

    def render(self) -> Layout:
        layout = self.layout()
        layout['banner'].update(self._banner())
        layout['role_panel'].update(self._role_panel())
        layout['stats'].update(self._stats_panel())
        layout['progress'].update(self._progress_table())
        layout['log'].update(self._event_log())
        layout['apps'].update(self._apps_footer())
        return layout

    def start_fake_progress(self):
        # Pick cli_app/api/library/default scaffold based on the req
        # and start animating per-file progress bars.

    def tick_fake_progress(self):
        # Advance one tick (~125ms). Caps at TICK_CAP=12 so all
        # bars finish in ~1.5s; after that the header shows
        # 'awaiting LLM response' instead of looping.
```

Key idea: LiveRunState is the shared mutable object between Team._run*_live and the rich Live display. Every state mutation triggers live.update(state.render()). The role panel has 4 phase-specific views: _starting_view (spinner + elapsed), _text_stream_view (PM/Architect/QA text), _fake_progress_view (Engineer file tree with progress bars), _real_manifest_view (final file tree).

metagpt_mini/cli.py - The `metagpt` console script (471 LOC)

The entry point declared in pyproject.toml. uv run metagpt invokes cli.main().

```
def main():
    args = sys.argv[1:]
    live = '--live' in args or not args      # default = live TUI
    if '--plain' in args: live = False
    if not args: return _run_demo(live=True) # the show!
    cmd = args[0]; rest = args[1:]

    if cmd in ('test', 'tests'): return test_cmd()
    if cmd == 'ping': return ping_cmd()
    if cmd == 'pdf': return _rebuild_pdf()
    if cmd == 'init': return _run_demo(' '.join(rest), live=True)
    if cmd in ('-h', '--help', 'help'):
        _print_help(); return 0

    # Otherwise treat as a free-form requirement
    return _run_demo(' '.join(args), live=live)

def _run_demo(requirement, *, live):
    llm = LLM()
```

```

team = Team(llm=llm, live=live)
result = team.run(requirement)
saved, test_result = _save_artifacts(result, ...)
_print_next_steps(saved, ...)

```

Key idea: `_save_artifacts` walks the result pool, pulls the latest PRD / Design / Manifest, and writes everything to output/ with a comment-header banner showing the run id, requirement, model, and generated-at timestamp. The generated project gets `pip install -e .` metadata in `pyproject.toml` and is auto-tested with `pytest` after the run.

pyproject.toml + .env - the bootstrap

`uv` is the only tool you need. `uv sync` reads `pyproject.toml`, creates `.venv/`, and installs every dep including the metagpt console script. `.env` holds your three secrets: `LLM_API_KEY`, `LLM_BASE_URL`, `LLM_MODEL`.

5. The SOP Trace - What You Will See Live

When you run `uv run metagpt`, here is the actual sequence of events your students will watch on the full-screen TUI. Numbered for ease of narration.

Step 1 - Full-screen TUI opens

The pyfiglet **MetaGPT-Mini** wordmark renders at the top. The Mission Control panel on the right shows the model (MiniMax-M3), endpoint, context window, and a token + cost counter at zero. The progress table shows all 4 roles as *pending*. The event log is empty.

Step 2 - User requirement enters as a Message

```
Message(role='User',
        content='Build a Python CLI todo app with add, list,
                complete, delete, priorities, due dates, and
                JSON persistence.',
        cause_by='UserInput',
        msg_id='a3f7c-...',
        round_num=0)
```

Step 3 - ProductManager runs WritePRD

The role panel becomes **cyan**, header reads *ProductManager . WritePRD*. Tokens stream in via `on_token` in real time. The progress table flips PM to *active*, then *done* when complete. Tokens-in / tokens-out / cost all tick up. The event log gains *ProductManager done (1,842 chars)*.

Step 4 - Architect runs WriteDesign

Same pattern, magenta. Reads the latest PRD from the pool and produces the design (modules, data model, interfaces, edge cases).

Step 5 - Engineer runs WriteCode - the big one

The role panel becomes green. Because `WriteCode` uses Anthropic tool-use (no streaming tokens), the panel first shows the **fake-progress view**: an animated file tree of 7 planned files (README, `pyproject.toml`, `src/main.py`, etc.) with progress bars ticking up file-by-file over ~1.5 seconds. The header then switches to *awaiting LLM response* with a Braille spinner and elapsed timer. When the structured call returns (~5-30s), the header reverts and the panel renders the **real manifest view**: project name, summary, file list with char counts, dependencies, run instructions.

```
# Example observable in the role panel after Step 5 lands:
#
# Engineer -> WriteCode  <green>done</green>
#
# cli_todo_app
#   A simple CLI todo app with persistent JSON storage.
#
#   README.md           - Project overview + run instructions  (462 chars)
#   pyproject.toml      - Package metadata + console-script    (311 chars)
#   src/__init__.py     - Package marker                       (40 chars)
#   src/main.py         - CLI entrypoint + argparse setup      (742 chars)
#   src/commands.py     - Subcommand handlers                 (923 chars)
#   src/storage.py      - JSON persistence layer               (511 chars)
#   tests/test_main.py  - pytest coverage for happy path      (815 chars)
#
# deps: stdlib
# run:  pip install -e . && python -m cli_todo_app
#
# Total: 7 files . 3,804 chars
```

Step 6 - QA runs WriteTest

Yellow panel. Reads design + the Engineer's manifest, calls `llm.structured(tool_name='qa_report')`. Returns `{passed: bool, issues: [], summary: ...}`. The progress table flips QA's status to *PASS* (green check) or *FAIL* (red x).

Step 7 - QA feedback loop (optional second round)

If QA fails, the Engineer is invoked again with `qa_feedback` in its system prompt (`CODE_REVISION_SYSTEM`). The Engineer regenerates *only the affected files*. Loop continues up to `MAX_REVISIONS=2` rounds (env-configurable). Each retry is shown as a new row in the progress table.

Step 8 - Artifacts written to disk

After the live TUI closes, `_save_artifacts` writes everything to `output/`:

- `output/01_prd.md` - the Product Requirements Document
- `output/02_design.md` - the technical design
- `output/<project_name>/` - the generated multi-file project (README, pyproject.toml, src/, tests/)
- `output/03_qa_report.md` - QA verdict + issues
- `output/04_run_summary.json` - run metadata

Step 9 - Auto-test the generated project

If the generated project contains `test_*.py` files, `_save_artifacts` runs `pytest` on them automatically and prints the result.

Step 10 - Next-steps hint

```
uv run metagpt
# ... TUI runs, artifacts saved ...

cd output/cli_todo_app
pip install -e .
python -m cli_todo_app --help
pytest -v                # run the generated test suite
```

If the SOP is correct, you have a working multi-file Python project built end-to-end by 4 LLM agents. Total wall time: 30-90 seconds with MiniMax-M3. Total cost: roughly **\$0.05 - \$0.20** per run.

6. Setup - The Two-Command Workflow

The whole toolchain is one Python package and one dependency manager. No Docker, no Node, no manual venv juggling. Works identically on macOS, Linux, and Windows (Git Bash or PowerShell).

6.1 Prerequisites

- **Python 3.11+** - any modern Python. Mac and Linux ship it; Windows users get it from python.org (check *Add to PATH*).
- **uv** - the single binary that replaces pip + venv + pip-tools. Install with `pip install uv`, `brew install uv`, or `winget install uv`.
- **LLM API key** - defaults to MiniMax-M3, but the framework is Anthropic-compatible so any provider works.

6.2 Clone + sync

```
# macOS / Linux / Git Bash on Windows
git clone https://github.com/mannuking/metagpt-mini.git
cd metagpt-mini
uv sync                                # creates .venv/ + installs deps + console script
```

6.3 Configure .env

```
cp .env.example .env
# or on Windows: copy .env.example .env

# Fill in your three secrets:
# LLM_API_KEY=sk-...
# LLM_BASE_URL=https://api.minimax.io/anthropic
# LLM_MODEL=MiniMax-M3

# .env is in .gitignore - NEVER commit it.
```

6.4 Verify + run

```
# Run the schema tests - no LLM required, runs in <1s
uv run metagpt test

# Open the live TUI (the show!)
uv run metagpt
```

6.5 Other commands

```
uv run metagpt                # Full-screen live TUI (default - the show!)
uv run metagpt --plain        # Blocking rich-panel output (good for piping)
uv run metagpt init 'req'     # Full-screen TUI with custom requirement
uv run metagpt 'your req'    # Same as above
uv run metagpt test           # 6 schema tests, no LLM
uv run metagpt ping           # Single LLM ping (proves API key works)
uv run metagpt pdf            # Regenerate this learning guide PDF
uv run metagpt help           # Show all options
```

6.6 Switching models without code changes

Because `llm.py` only talks the Anthropic Messages protocol, switching providers is a `.env` change:

```
# MiniMax-M3 (default)
LLM_BASE_URL=https://api.minimax.io/anthropic
LLM_MODEL=MiniMax-M3

# Anthropic Claude (direct)
LLM_BASE_URL=https://api.anthropic.com
LLM_MODEL=claude-3-5-sonnet-latest
```

```
# Local Ollama (Anthropic-compatible mode)
LLM_BASE_URL=http://localhost:11434
LLM_MODEL=qwen2.5-coder:7b
LLM_API_KEY=ollama           # any non-empty string
LLM_CONTEXT_WINDOW=32768     # Ollama models have smaller context
```

6.7 Common gotchas

- **.env not loaded:** python-dotenv reads .env from the current working directory. Always cd into the project folder before running.
- **API key leakage:** if you accidentally commit .env, rotate the key immediately. GitHub push protection also auto-redacts known provider key patterns.
- **Slow first call:** the first LLM call may take 5-10s for TLS handshake + cold-start; subsequent calls are 1-3s.
- **TUI in tiny terminals:** the live TUI needs ~100x30 minimum. Maximize the window first.
- **uv hangs on first sync:** if uv sync hangs, check your Python version (python --version >= 3.11).

7. The Live TUI - Architecture Deep-Dive

The full-screen TUI is the centrepiece of the class demo. This section explains how it is built, why each region exists, and how it stays alive while the LLM is thinking.

7.1 The 6-region layout

The screen is split vertically into 5 regions; the middle *main* region is itself split horizontally into 2:

```
+-----+
| banner          (pyfiglet 'MetaGPT-Mini' + ICLR footer) | size=10
+-----+
| role_panel      ratio=2, min=40 | stats          size=38 |
| (current role streaming)         | (model, ctx %, tokens, |
|                                   | cost, throughput,    |
|                                   | per-role breakdown)  |
+-----+
| progress        (4 rows: role, action, round, status)   | size=5
+-----+
| log             (scrolling event log, color-coded, ts)   | size=10
+-----+
| apps            (real-world SOP-style agents: MetaGPT,    | size=3
|                 Cursor, Devin, AutoGen, CrewAI, LangGraph) |
+-----+
```

7.2 State + render split

LiveRunState is the single mutable state object shared between Team._run*_live and the rich.live.Live context. Every state mutation is followed by live.update(state.render()). The render() method calls layout() to build the 6-region skeleton, then populates each region with a phase-specific view.

7.3 The role panel has 4 phase-specific views

- **_starting_view**: animated Braille spinner + role name + profile + *working... N.Ns elapsed*. Active during the LLM preflight gap before the first token arrives.
- **_text_stream_view**: rolling text buffer of the last 200 lines of streamed tokens, with a ... *[N earlier lines]* ... indicator when truncated. Used for PM, Architect, QA.
- **_fake_progress_view**: animated file tree of 7 planned files with per-file progress bars ticking up file-by-file. Picks cli_app / api / library / default scaffolding based on the requirement keywords. Caps at 1.5s of motion; then the header switches to *awaiting LLM response* with the elapsed timer still ticking.
- **_real_manifest_view**: final file tree from the manifest: project name, summary, file list with char counts, dependencies, run instructions, total chars. Fills the entire role panel.

7.4 The worker-thread pattern

The critical design choice: each _run_role_live / _run_engineer_live / _run_qa_live spawns the actual LLM call in a **daemon worker thread**. The main thread then loops on worker.join(timeout=0.1) - 100ms ticks - advancing the spinner / fake-progress bars and calling live.update(state.render()) each time. When the worker finishes, the state is updated synchronously and the loop exits. This means the TUI **never blocks**: every LLM call - 100ms or 30s - has visible UI motion.

7.5 The Mission Control stats panel

Right column. Pulls from llm.usage on every refresh:

- **Model + endpoint** - read from env at startup
- **Context window** - *tokens_in* / *context_window* (X.X%), live progress bar in your head
- **Tokens in / out / total** - running counters
- **Cost so far** - green USD estimate using *COST_INPUT_PER_1K* / *COST_OUTPUT_PER_1K*
- **Budget** - if *MAX_BUDGET_USD* is set, show remaining
- **Throughput** - tokens / second since the run started
- **Per-role breakdown** - splits tokens + cost by role (ProductManager / Architect / Engineer / QA)

8. The QA Feedback Loop - Engineer Gets a Second Chance

MetaGPT's full SOP is not strictly linear. After the first Engineer+QA round, if QA fails, the Engineer is re-invoked with QA's feedback. This is a simplified Reflexion-style verbal refinement loop - and it is what brings us from 'demo that produces code' to 'demo that produces code that passes tests'.

8.1 How the loop fires

In `Team._run_with_live`:

```
for round_num in range(1, max_revisions + 2): # 1..max_revisions+1
    qa_feedback = self._last_qa_issues_text() if round_num > 1 else None
    self._run_engineer_live(roles[2], round_num, qa_feedback, live=live)
    qa_msg = self._run_qa_live(roles[3], round_num, live=live)
    result.qa_passed = bool(qa_msg.extra.get('passed', False))
    if result.qa_passed:
        break
    # else: loop again, Engineer sees the new feedback
```

Default `MAX_REVISIONS=2` (env-configurable). That means up to 3 total rounds: round 1 fresh, round 2 with feedback, round 3 with round-2 feedback.

8.2 What the Engineer sees on round 2

`WriteCode.run()` takes an optional `qa_feedback` argument. When non-None, it switches from `CODE_SYSTEM` to `CODE_REVISION_SYSTEM`:

```
CODE_REVISION_SYSTEM = (
    'You are a senior Backend Engineer. The QA agent found issues\n'
    'in your previous output. The design is below. Read the QA report\n'
    'carefully and regenerate the affected files ONLY (return the full\n'
    'content of each revised file). If a file is unchanged, omit it.\n'
    'Be minimal: do not rewrite files that are correct.'
)

user_text = (
    f'Design:\n{design.content}\n\n'
    f'QA feedback from previous round:\n{qa_feedback}\n\n'
    'Regenerate the affected files. Return only files that changed.'
)
```

The Engineer regenerates *only the failing files* - much cheaper than a full rewrite, much more likely to converge.

8.3 What the QA produces

`WriteTest` uses `llm.structured(tool_name='qa_report')` with this schema:

```
QA_REPORT_SCHEMA = {
    'type': 'object',
    'properties': {
        'passed': {'type': 'boolean'},
        'issues': {'type': 'array', 'items': {'type': 'string'}},
        'summary': {'type': 'string'},
    },
    'required': ['passed', 'issues', 'summary'],
}
```

We force conservatism: if QA returns `passed=true` but lists 2+ issues, we flip it to `passed=false`. This keeps the loop honest.

8.4 Where this maps in the paper

The original MetaGPT SOP is exactly this: PM and Architect run once, Engineer and QA iterate. We borrow that structure. The verbal self-reflection idea (Reflexion) lives implicitly in CODE_REVISION_SYSTEM: QA's issues are the 'reflection', the Engineer's revised files are the 'improved trial'. We do not yet persist reflections across runs (that would be the true Reflexion extension).

8.5 Observed behaviour in class

Round 1: Engineer produces a 7-file CLI todo app. QA flags: *storage.py does not handle corrupt JSON*. Round 2: Engineer rewrites `src/storage.py` only (plus a new `tests/test_storage.py`). QA passes. Total wall time: ~80s. Total cost: ~\$0.12. The progress table shows the verdict flip live.

Appendix A - Glossary for the Class

Agent - An LLM-driven entity that reads context, decides, and acts. In MetaGPT, a Role is an Agent.

Action - The unit of behaviour. An Action takes context and emits a Message.

Role - A named bundle of Actions + a profile. Roles are reusable across runs.

SOP - Standard Operating Procedure. The directed graph of Actions that defines a workflow.

Message - An event emitted by an Action. Carries role, content, cause_by, msg_id, timestamp, round_num, extra dict.

Message Pool - The shared bus through which agents communicate.

Manifest - The structured multi-file project output from WriteCode. Carries project_name, files[], deps, run_instructions.

Structured Output - Tool-use API call (Anthropic Messages tool block) that forces the LLM to return JSON matching an input_schema. No parsing roulette.

ACI - Agent-Computer Interface (SWE-agent term). Purpose-built verbs instead of raw shell.

Harness - The runtime + tooling + environment around an LLM. MetaGPT is a harness; OpenHands is a harness; Hermes Agent is a harness.

Self-Reflection - An agent's own critical assessment of its output (Reflexion idea). In MetaGPT-Mini this is implicit in the QA round's issues list.

Skill Library - A growing collection of reusable code/tool definitions an agent writes and reuses (Voyager idea).

uv - The single Python tool that replaces pip + venv + pip-tools. Reads pyproject.toml, creates .venv, installs deps, builds console scripts. One binary, written in Rust.

Live TUI - Full-screen rich.live display. The 6-region layout shown in the class demo.

Fake Progress - Animated file tree shown while Engineer is in its structured tool-use call. No real tokens stream there, so we synthesize a realistic scaffold.

Appendix B - Citations (BibTeX)

```
@inproceedings{hong2024metagpt,
  title = {MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework},
  author = {Hong, Sirui and Chen, Jonathan and Zhu, Ceyao and others},
  booktitle = {ICLR},
  year = {2024},
  note = {Oral presentation, top 1.2 percent, ranked no.1 in LLM-Agent category}
}

@inproceedings{yang2024sweagent,
  title = {SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering},
  author = {Yang, John and others},
  booktitle = {NeurIPS},
  year = {2024}
}

@inproceedings{wang2024openhands,
  title = {OpenHands (formerly OpenDevin): An Open Platform for AI Software Developers},
  author = {Wang, Xingyao and others},
```

```

    booktitle = {COLM},
    year = {2024},
    note = {Oral presentation}
}

@inproceedings{shinn2023reflexion,
  title = {Reflexion: Language Agents with Verbal Reinforcement Learning},
  author = {Shinn, Noah and Cassano, Federico and Gopinath, Ashish and others},
  booktitle = {NeurIPS},
  year = {2023}
}

@inproceedings{wang2023voyager,
  title = {Voyager: An Open-Ended Embodied Agent with Large Language Models},
  author = {Wang, Guanzhi and Xie, Yuqi and Jiang, Yunfan and others},
  booktitle = {NeurIPS},
  year = {2023}
}

```

Appendix C - File-by-file cheat sheet

Total: **2,408 LOC of Python** in metagpt_mini/. Plus ~160 LOC of scripts and ~50 LOC of tests.

File	LOC	What it does
metagpt_mini/__init__.py	1	Package marker + version.
metagpt_mini/llm.py	221	Anthropic Messages client. chat/stream/structured + LLMUsage cost tracker.
metagpt_mini/schema.py	82	Message, MessagePool, Manifest, FileEntry, run_id. The substrate.
metagpt_mini/actions.py	443	WritePRD, WriteDesign, WriteCode (tool-use), WriteTest (tool-use).
metagpt_mini/roles.py	55	Role + make_canonical_team. 4 roles, 4 colors.
metagpt_mini/team.py	511	Orchestrator + worker-thread live-mode runner + QA feedback loop.
metagpt_mini/ui.py	623	Full-screen TUI: 6 regions, 4 phase views, fake-progress animation.
metagpt_mini/cli.py	471	`metagpt` console script: test, ping, pdf, init, --plain, --live.
examples/build_cli_app.py	80	Blocking-mode demo (legacy; use `uv run metagpt` instead).
scripts/ping.py	~30	Single LLM ping. Proves API key + endpoint reachability.
scripts/test_one_action.py	~50	Run one action (e.g. WritePRD only) in isolation.
scripts/full_run.py	~80	Full SOP run with timing breakdown per role.
tests/test_schema.py	~50	6 unit tests. Run with `uv run metagpt test`.
papers/01_swe_agent.pdf	-	SWE-agent (NeurIPS 2024).
papers/02_metagpt.pdf	-	MetaGPT (ICLR 2024 Oral) - the one we reimplemented.
papers/03_openhands_opendevin.pdf	-	OpenHands (COLM 2024 Oral).
papers/04_reflexion.pdf	-	Reflexion (NeurIPS 2023).
papers/05_voyager.pdf	-	Voyager (NeurIPS 2023).
papers/REVIEW.md	-	Comparison + selection justification across all 5 papers.
pyproject.toml	-	uv-managed, single source of truth. Console-script entrypoint lives here.
README.md	-	GitHub README. Top-level orientation.
.env.example	-	Copy to .env, fill in LLM_API_KEY + LLM_BASE_URL + LLM_MODEL.
docs/build_pdf.py	-	Generates this PDF.

docs/MetaGPT-Mini-Learning-Guide.pdf	-	This document.
--------------------------------------	---	----------------